TraceDAE: Trace-Based Anomaly Detection in Microservice Systems via Dual Autoencoder

Junjun Li, Shi Ying, Tiangang Li, and Xiangbo Tian

***Abstract*—Microservice systems have become a popular architecture for modern Web applications owing to their scalability, modularity, and maintainability. However, with the increasing complexity and size of these systems, anomaly detection emerges as a critical task. In this paper, we introduce TraceDAE, a trace-based anomaly detection approach in microservice systems. The approach initially constructs a Service Trace Graph (STG) to depict service invocation relationships and performance metrics, subsequently introducing a dual autoencoder framework. In this framework, the structure autoencoder employs Graph Attention Networks (GAT) to analyze the structure, while the attribute autoencoder leverages the Long Short-Term Memory Network (LSTM) for processing time series data. This approach is capable of effectively identifying Service Response Abnormal and Service Invocation Abnormal. Moreover, the final experimental results on datasets show that TraceDAE is an efficient anomaly detection approach which outperforms the SOTA(State of The Arts) trace-based anomaly detection methods with F1-scores of 0.970 and 0.925, respectively.**

***Index Terms*—Microservice systems, anomaly detection, trace analysis, dual autoencoder.**

I. Introduction

MICROSERVICE systems have become a popular architecture for modern Web applications attributable to their scalability, modularity, and ease of maintenance [1], [2], [3]. These architectures decompose services into diminutive, independently deployable services that communicate over the network, thereby offering complete application functionality. Even though a lot of efforts are devoted to quality assurance of microservice systems, owing to their increasing size and complexity, microservice systems often seem fragile and the risk of system failures increases gradually [4], [5]. Once a failure occurs, it may precipitate service disruption, potentially causing significant financial losses to the organization and undermining user satisfaction. For instance, on the evening of November 27, 2023, a system failure at DiDi lasted for nearly 12 hours, causing a large number of users unable to locate and take taxis. This malfunction resulted in direct losses for DiDi exceeding 10 million orders, with a transaction volume of approximately ¥400 million [6]. Consequently, anomaly detection and root cause localization of microservice systems are of great significance. They not only help us report system faults accurately but also provide valuable guidance for subsequent fault analysis.

Anomaly detection and root cause localization in microservice systems have garnered significant attention over the years [7], [8], [9], [10], [11]. Currently, most anomaly detection methods primarily concentrate on logs and metrics [12], [13], [14], which often make it difficult to accurately determine the type of anomaly and support subsequent root cause localization. However, in microservice systems, there are multiple types of data sources, among which trace data carries rich attribute information [15]. Among existing trace-based approaches, a common practice is to convert trace data into numeric vectors, which enables the application of machine learning techniques. Some approaches, notably [16] and [17], perform analysis by defining features such as the frequency of different business operations. Other approaches emphasize the structural features of the invocation paths, transforming the metric data on these paths into vectors.

Unfortunately, most existing trace-based approaches ignore the rich detailed information carried by trace data. We strongly believe that fully utilizing the detailed information is crucial for achieving more accurate anomaly detection and finer root cause localization [18].

In this paper, we propose TraceDAE, an anomaly detection method based on trace data in microservice systems. This method is based on trace data and uses metric data as built-in attributes to construct a Service Trace Graph (STG). STG not only reflects the invocation relationship between services but also incorporates performance metrics for each invocation. Based on STG, we design two integrated autoencoders: structure autoencoder and attribute autoencoder, each focusing on specific aspects of the trace data. The structure autoencoder analyzes the structural information of the STGs through Graph Attention Networks (GAT) to capture the dependencies between services. The attribute autoencoder, on the other hand, employs Long Short-Term Memory Network (LSTM) to process time-series data to identify anomalous changes in performance metrics over time. By integrating the loss values of these dual autoencoders, TraceDAE can effectively identify anomaly traces and their anomaly types in a microservice system. For the anomaly trace data, we compute the anomaly score for each microservice to locate the root cause of the anomaly traces. We conducted extensive

Received 29 April 2024; revised 20 September 2024; accepted 23 June 2025. Date of publication 26 June 2025; date of current version 7 October 2025. This work was supported by the National Natural Science Foundation Project of China under Grants 62472329, 62072342, and 61672392, and by the National Key Research and Development Plan Project of China under Grant 2022YFB3304300. The associate editor coordinating the review of this article and approving it for publication was M. F. Zhani. *(Corresponding author: Shi Ying.)*

The authors are with the School of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: lijunjun@whu.edu.cn; shiying@whu.edu.cn; tiangangli@whu.edu.cn; tianxiangbo@whu.edu.cn).

Digital Object Identifier 10.1109/TNSM.2025.3583213

1932-4537 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

experiments to compare the performance of six SOTA(State of The Arts) trace anomaly detection methods based on Train-Ticket [19], an open-source microservice benchmark, as well as a large-scale Internet service provider's production microservice system. The experimental results show that the TraceDAE performs excellently in F1-scores, reaching 0.970 and 0.925 respectively, which are 0.235 and 0.11 higher than the highest scores of the baseline method. Our contribution can be summarized as follows:

1. We propose a Service Trace Graph (STG) construction method. It integrates trace and metrics into a graph to represent microservices and their invocations, thus providing a comprehensive view of system operations and interactions.

2. We design a dual autoencoder architecture that combines structure autoencoder and attribute autoencoder, and utilizes Graph Attention Network (GAT) and Long Short Term Memory Network (LSTM) to capture structural and attribute features of trace data, respectively. In this way, we can effectively detect response time anomalies and invocation path anomalies in microservice invocations.

3. We design and implement TraceDAE and conduct extensive experiments using two different datasets. The experimental results show that our approach performs well in microservice trace anomaly detection, with average F1-scores of 0.970 and 0.925 respectively, which are 0.235 and 0.119 higher than the most advanced methods on average.

The remainder of this paper is organized as follows. Section II provides the related work. Section III further introduces some basic concepts and the problem definition. Section IV presents our approach. Section V discusses the experimental setup and presents the results of our evaluations. Section VI concludes the paper, summarizing key contributions and suggesting avenues for future research.

II. Related Work

Anomaly detection in microservice systems is an important research field [20], [21]. Many related research works have been proposed. Depending on the type of data and techniques used, these works can be categorized into different classes. In this section, we provide a comprehensive review of related works and discuss them in detail.

A. Anomaly Detection Based on Logs and Metrics

Some of the early-stage anomaly detection methods were based on logs and performance metrics. Meng et al. proposed LogAnomaly [12], a framework for modeling a log stream as a natural language sequence. Utilizing template2vec to extract the semantic information hidden in log templates, LogAnomaly can detect both sequential and quantitative log anomalies. Li et al. proposed SwissLog [13], a deep learning-based anomaly detection model. This model introduces a log parser that combines semantic embedding and temporal embedding to train a unified attention-based BiLSTM model for solving problems caused by log order changes and log time interval changes. Zhang et al. proposed NeuralLog [14], a log anomaly detection method without parsing. The method extracts raw log messages to represent them as semantic vectors and then detects the semantic items by Transformer's classification model, which captures contextual information from log sequences. He et al. proposed a method TopoMAD [22] for anomaly analysis detection using spatiotemporal deep learning. The method collects data such as CPU, memory, disk, and network metrics from containers to capture temporal and spatial dependencies. The method utilizes graph neural networks to extract spatial features and LSTM networks to extract temporal features. Zhang et al. proposed a time series anomaly detection model TFAD [23] based on time-frequency analysis, which incorporates time series decomposition and data expansion mechanism in the time-frequency architecture to improve the performance and interpretability of anomaly detection. Yang et al. proposed MicroMILTS [24], a microservice fault localization method based on mutual information and LSTM autoencoder, which uses the BIRCH clustering algorithm to analyze performance metrics associated with microservice anomalies for anomaly detection. Although these methods have good performance in anomaly detection for microservices, most of them ignore the trace data that is related to the invocation information that can react to more microservice system interactions.

B. Anomaly Detection Based on Traces

With the emergence of trace data in microservice systems, some research work has been conducted to perform anomaly detection based on trace data. Early research converted traces into numerical vectors and then used machine learning techniques for anomaly detection. Las-Casas et al. proposed a method [16] based on execution graph clustering for the overhead and storage requirements of traces, which can bias the requested traces, and improve the storage problem of storing the diversity of traces in infrequent patterns. Fu et al. constructed a finite state automaton FSA [25] to learn the time distribution and detect anomalies k-sigma based on trace data. The above methods only encode trace data as a numerical value, without utilizing the complex structural information of trace. Nedelkoski et al. proposed Multimodal-LSTM [26], a deep learning model based on time series learning, which uses LSTM networks to construct a multimodal architecture to detect microservice anomalies using structural features of response time and trace. Liu et al. proposed TraceAnamoly [27], a trace anomaly detection method based on a deep Bayesian network that encodes traces as STVs and uses VAE to detect anomalous traces. Zhang et al. proposed CRISP [28], which represents traces as STV(Service Trace Vector) and detects trace anomalies through VAE reconstruction. These methods encode trace as a vector, which although utilizing the structural information of trace, still do not fully exploit the potential of the complex attribute information carried by trace.

In recent years, several research works have focused on the structural properties of traces, transforming them into tree or graph structures. Deep learning algorithms have been used to learn the call characteristics between traces in depth to enable anomaly detection.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

Meng et al. [29] and Wang et al. [30] proposed a method to detect microservice anomalies by comparing execution traces, which calculates trace anomalies by tree edit distance and detects response time anomalies by principal component analysis. TraceLingo [31] and DeepTraLog [32] use LSTM and GNN methods, respectively, to capture the tree structure in the trace to obtain an accurate representation of the trace. TraceLingo requires supervised training, while DeepTraLog uses additional logging data. TraceVAE [33] encodes traces as graph structures and proposes a solution for graph VAE. Zhang et al. proposed a comparative representation learning method, TraceCRL [34], which represents trace data as a graph structure, and makes similar traces closer in the representation space through representation learning for more accurate and efficient anomaly detection and fault diagnosis.

Although these methods fully consider the structural information of trace, they do not integrate the attribute information of trace to comprehensively achieve anomaly detection and further analyze the types of anomalies in the results of anomaly detection.

C. Root Cause Localization Based on Traces

Trace describes the invocation relationships between services in a microservice system, which provides an important basis for fault root cause localization methods. Liu et al. proposed a microservice system root cause of failure localization method MicroHECL [35], which analyzes potential anomaly propagation chains based on service invocation graphs and ranks candidate service instances based on correlation analysis to discover a ranked sequence of potentially faulty microservice instances. Yu et al. proposed MicroRank [36], a root-cause localization method based on the extended-spectrum technique, which identifies the root cause of a fault by pre-training features in normal and abnormal trace data. Zhou et al. proposed an approach MEPFL [17] for fault localization in microservice applications, which predicts potential faults by analyzing the trace logs of the application. Lin et al. proposed an approach MicroScope [37] for fault localization in microservice applications, which uses directed acyclic graphs to represent dependencies between microservices. Li et al. proposed TraceRCA [38], which uses unsupervised machine learning to detect various metrics over the same period in a trace to identify anomalous traces. For anomalous traces, TraceRCA calculates a suspicion score by passing in and outputting the number of traces of anomalous calls and ranks all microservices. Although these methods utilize trace structure for root cause localization, they do not take full advantage of the feature information contained in the traces.

III. Problem Definition

In this section, we outline the service invocation relationships in microservice systems and provide a formal definition of the trace exception detection problem for microservice systems. Table I enumerates the relevant notations and their descriptions, providing a basis for the subsequent discussion.

We define the microservice system as an interconnected network of services $S = \{s_1, s_2, \ldots, s_N\}$, where each service s_i interacts with others through network calls. These interactions generate trace data, which can be represented as a sequence of traces $T = \{(t_1, s_1, s_2, x_1), (t_2, s_2, s_2, x_2), \ldots, (t_i, s_i, s_j, x_i)\}$, where t_i denotes the timestamp of the ith interaction, s_i, s_j signifies the microservice s_i invoking the microservice s_j, and x_i represents a high-dimensional vector capturing the features extracted from that interaction, including but not limited to the source and target services and response times.

To illustrate this with an example, consider a simple e-commerce microservice system where the user authentication service s_1 calls the user profile service s_2 to retrieve user details. This interaction would be logged as a trace like (t_1, s_1, s_2, x_1), where t_1 is the timestamp of the request, and x_1 could include features like the response time of the user profile service, the size of the response, and the success or failure status of the interaction.

Our core goal is to design and develop a model TraceDAE for identifying anomalous trace T in microservice systems S. Here, an anomaly is characterized as a significant deviation from the normal mode of operation of a microservice system, indicative of a potential problem or failure. For instance, if the response time of a service, such as the payment processing service in an e-commerce system, suddenly increases beyond its typical range, this could signal an issue such as network latency, resource exhaustion, or a software bug. TraceDAE aims to detect such deviations automatically, helping to maintain the system's reliability and performance.

In order to better represent the invocation relationship between services, TraceDAE constructs the trace data T as a Service Trace Graph $STG = \{V, E, X\}$, where V represents the set of services involved in the trace acting as nodes in the graph, E denotes the service invocations, acting as edges connecting the nodes, and X denotes the latency and performance metrics generated when the service is invoked, which is also referred to as the service attribute matrix. The STG is formally a graph with M nodes, each associated with an N - dimensional attribute vector, where N represents the number of features used to characterize the interactions.

In the anomaly detection phase, TraceDAE encodes the STG by a dual autoencoder, then computes its reconstruction loss function value. More formally, we aim to learn a loss function $L : T_i \longmapsto y_i \in \mathbb{R}$, to classify the sample T_i based on the threshold λ:

$$y_i = \begin{cases} 1, \text{ if } L(T_i) \geq \lambda \\ 0, \text{ otherwise} \end{cases} \tag{1}$$

where y_i denotes the label of sample trace T_i, with 0 being the normal class and 1 being the anomalous class.

For abnormal traces, in more detail, we classify them into two categories based on their different characteristics Service Response Abnormal and Service Invocation Abnormal.

- *Service Response Abnormal:* It means that the duration of the response to a user request deviates significantly from the normal value, either being too long or abnormally short. This may occur, for example, when an application faces resource constraints due to a large number of requests or irregular resource utilization patterns.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMMONLY USED NOTATIONS

Notations	Descriptions
S	The set of services in the microservice system.
T	The trace record generated by a microservice invocation after a request
STG	A graph representing a service trace.
V	The set of nodes in a STG.
E	The set of edges in a STG.
X	The set of attributes in the service node.
N	The numbers of nodes(V) in a STG.
M	The dimensionality of the attribute X in a STG.
A	The adjacency matrix of a graph.
$\hat{A}$	The reconstruction of A.
X	The feature matrix of a graph.
$\hat{X}$	The reconstruction of X.
L	The loss of reconstruction.

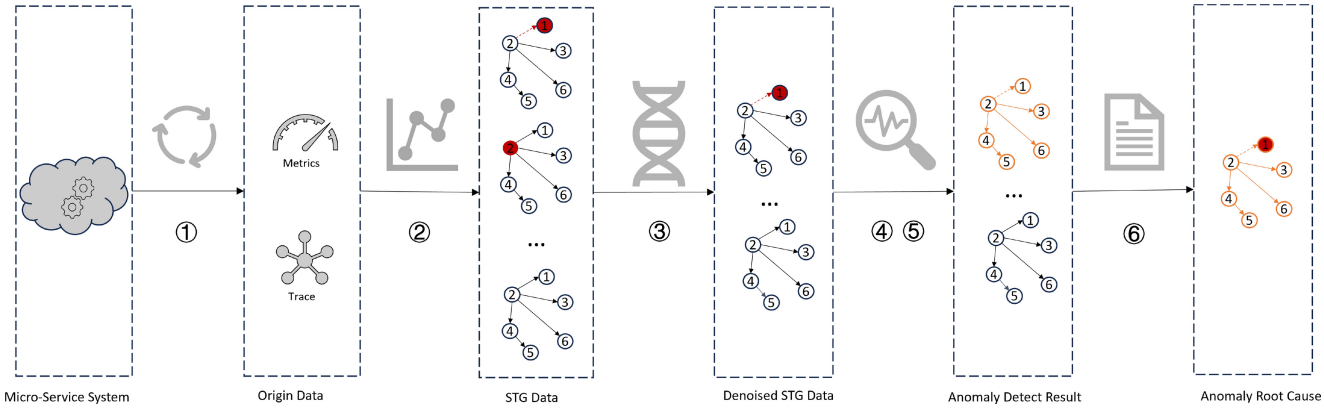


Fig. 1. The overview of TraceDAE workflow.

- *Service Invocation Abnormal:* It denotes a significant change in the execution path of a trace. This may occur when there are noticeable deviations in the sequence of microservice invocations. For instance, errors within certain microservices, such as network disruptions, can result in the interruption of user requests, leading to changes in the expected invocation path.

By categorizing anomalies in this way, we can more precisely diagnose the root cause of potential issues in microservice systems, leading to more targeted and effective troubleshooting.

IV. APPROACH

A. Overview

In this section, we present the TraceDAE, a trace-based anomaly detection method in microservice systems via dual autoencoders, which can efficiently detect trace anomalies. Figure 1 illustrates the overall workflow of the method, which can be concluded by several steps as follows:

- *Step 1: Collecting origin data.* microservice systems generate a continuous stream of raw data, encompassing trace and metric data. The trace data describes the invocation dependencies between microservices, and the metric data is the performance indicator data generated during the service invocation.
- *Step 2: Building STG.* Based on the collected trace and metric data, we construct them as a Service Trace Graph STG to represent the relationship of microservice invocations.
- *Step 3: Noise reducing.* To sanitize the dataset and eliminate unwanted noise, we employ density-based DBSCAN clustering [39]. This technique effectively filters out outliers, ensuring the accuracy and reliability of subsequent analysis.
- *Step 4: Training model.* During the training phase, the model learns to extract relevant features from different data modalities and encode their interactions effectively. It then uses attention mechanisms to assign weights to different features based on their relevance and importance for detecting anomalies.
- *Step 5: Abnormal detection.* For each STG, we calculate an anomaly score. If this score surpasses a predefined threshold γ, the STG is marked as an outlier, indicating potential abnormalities.
- *Step 6: Root cause localization.* For each abnormal STG, we compute anomaly scores for each node. The node with the highest anomaly score among the top k nodes is identified as the likely root microservice responsible for the anomaly.

B. Building STG

Microservice systems accurately reflect their operational status through various types of raw data. As shown in Fig. 2, we propose a method to convert service invocation

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

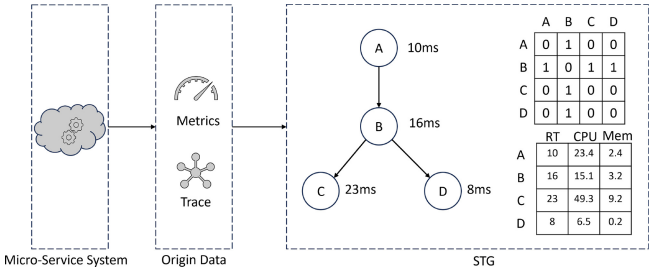


Fig. 2. The overview of building the Service Trace Graph (STG).

relationships in the microservice system into a unified graph representation called the Service Trace Graph (STG). This is a crucial step for effectively applying TraceDAE to anomaly detection. The STG is designed to capture the complex interactions and dependencies between microservices, including both the invocation relationships and the performance metrics of the services, which are essential for subsequent analysis and processing. The following section provides a detailed explanation about how to build a STG:

1) *Collecting Trace Data.* For a microservice system, there are hundreds of microservices. We collect trace data and temporal performance data generated during service invocation and store them in a temporal database.
2) *Preprocessing Trace Data.* For the collected trace data, we normalize the data to ensure consistency. When a service generates trace data in different formats, we preprocess it to ensure that each user request generates a unique Trace Id, and each service invocation generates a unique Span ID.
3) *Constructing STG Nodes.* For each trace record, we construct the unique Span ID generated by the service invocation as a node in the STG. Different from previous approache TraceCRL [34], where nodes represent service operations, in our STG, nodes represent entire services. Moreover, the temporal performance data generated by the service invocation is attached as attributes of the node, which include metrics such as timestamp, average response time, CPU usage, and memory usage. This structure allows the STG to encapsulate a richer set of information compared to traditional operation invocation graphs.
4) *Constructing STG Edges.* For each trace record, we construct service invocation dependencies from the source service node to the target service node as edges in the STG. In contrast to the edges in TraceCRL's operation invocation graph [34], which also encodes operation states and predefined attributes, our STG focuses on representing the invocation relationships at the service level, with the associated performance metrics embedded within the nodes rather than the edges.

TraceDAE has made significant breakthroughs in handling service invocation relationships in microservice systems with its unique four core steps. These steps cleverly transform the originally intricate invocation relationships into a clear and structured form, thereby constructing a unified service trace graph (STG) as the core graph representation. Unlike traditional graph representation learning techniques such as TraceCRL [34] utilizing DeepWalk [40] traversal operations on dependency graphs, TraceDAE introduces an innovative tree based graph construction algorithm specifically optimized for the STG construction process. This Algorithm 1 not only significantly improves the generation efficiency and quality of STG, but also further highlights the unique advantages and innovative significance of STG in anomaly detection and analysis of microservice systems.

Algorithm 1 Constructing the Service Trace Graph (STG)

Input: Trace data T, containing trace records; each trace record consists of service invocations with Span ID, Trace ID, and temporal performance data CPU Mem Time.;
Output: $STG = (V, E)$;

```
Initialize an empty graph STG = (V, E)
for each trace record r in T do
    Initialize an empty list nodesList ← []
    for each service invocation s in r do
        Extract or generate unique Span ID for service invocation s
        Extract temporal performance data associated with s
        if Span ID of s is not in V then
            Create a new node v with Span ID as identifier
            Attach temporal performance data to v
            Add v to the graph STG
        end if
        Append v to nodesList
    end for
    for each consecutive pair of nodes (v_source, v_target) in nodesList do
        if edge (v_source, v_target) is not in E then
            Create a new directed edge e from v_source to v_target
            Add e to the graph STG
        end if
    end for
end for
return STG.
```

C. TraceDAE

The TraceDAE model is an unsupervised joint learning framework designed to address the problem of trace anomaly detection in microservice systems. As shown in Fig. 3, TraceDAE achieves a comprehensive analysis of microservice trace data by fusing two core components, the structure autoencoder and the attribute autoencoder. The structure autoencoder captures the structural information in the microservice system through the graph autoencoder (GAE). The attribute autoencoder focuses on processing temporal data, i.e., changes in attributes, through the LSTM autoencoder (LSTM-AE). By integrating the loss values of these two components as the anomaly score, the TraceDAE model is able to accurately detect trace response times and invocation path anomalies. Next, we describe in detail the design of these two autoencoders and how they are integrated into a unified model.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

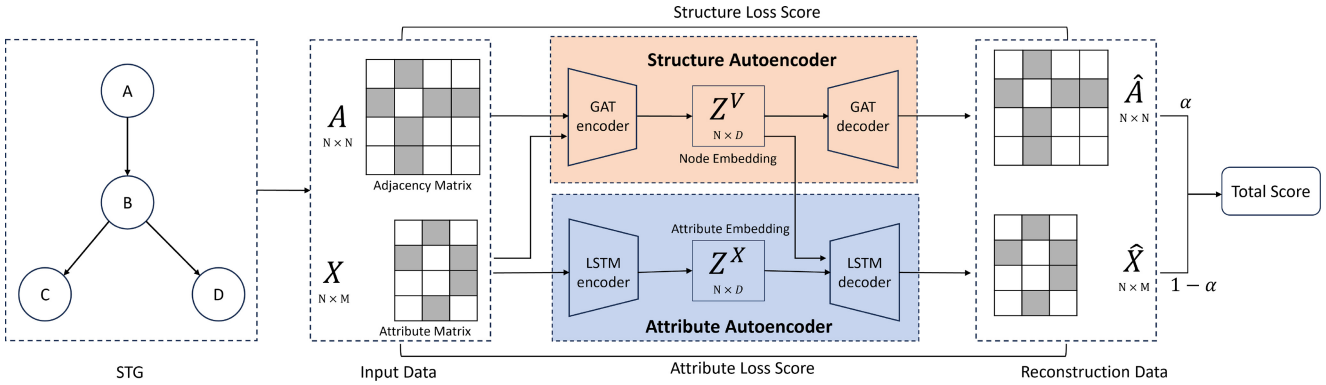


Fig. 3. The architecture of TraceDAE.

1) Structure Autoencoder: The core of a structure autoencoder lies in leveraging attention mechanisms to learn the structural relationships between service nodes and their neighbors, aiming to capture effective structural patterns. This model comprehends the contextual surroundings of a service node within the entire microservice architecture. By adopting an attention mechanism, the structure autoencoder evaluates the relative significance among nodes and their neighbors, facilitating the learning of impactful structural patterns.

To enhance the dimensionality of node features, the structure autoencoder initially transforms the originally observed node attribute X into a lower-dimensional latent representation, denoted by $\widetilde{Z}^V$. This transformation is defined as:

$$\widetilde{Z}^V = \sigma(XW_x + b_x) \tag{2}$$

where $\sigma(\cdot)$ represents an activation function, such as *ReLU* or *Tanh*. Here, $W_x \in R^{M\times D}$ and $b_x \in R^D$ are the weight matrix and bias vector learned by the encoder, *M* and *D* are the dimensionalities of *X* and $\widetilde{Z}^V$, respectively.

After performing the transformation, we treat $\widetilde{Z}^V$ as a graph attention layer with node eigenvalues of $\widetilde{Z}^V = \{\widetilde{Z}_1^V, \widetilde{Z}_2^V, \ldots, \widetilde{Z}_N^V\}$, where *N* is the number of nodes. We perform a shared attention mechanism on the nodes, employing graph attention layers to aggregate representations from neighboring nodes:

$$e_{i,j} = attn\left(\widetilde{Z}_i^V, \widetilde{Z}_j^V\right) \tag{3}$$

where $e_{i,j}$ denotes the importance weight of the node V_i relative to node V_j. The function $attn(\cdot)$ signifies a weight that is uniformly applied across all nodes. Here, we use *LeakyReLU* function. Consequently, $e_{i,j}$ can be described as follows:

$$e_{i,j} = attn\left(\widetilde{Z}_i^V, \widetilde{Z}_j^V\right) = LeakyReLU\left(\alpha^T\left[W\widetilde{Z}_i^V || W\widetilde{Z}_j^V\right]\right) \tag{4}$$

where $\alpha \in \mathbb{R}^D$ and $W \in \mathbb{R}^{\frac{D}{2}\times D}$ represent the neural network weights that parameterize the attention mechanism, shared across all nodes. The operator $||$ denotes concatenation. Then, *softmax* is used to normalize the neighboring nodes of the central node:

$$\gamma_{i,j} = softmax\left(e_{i,j}\right) = \frac{\exp\left(e_{i,j}\right)}{\sum_{k\in N_i}\exp\left(e_{i,k}\right)} \tag{5}$$

where N_i represents the set of neighbors for the node h_i provided by the adjacency matrix A. The final embedded representation Z_i for node h_i is then derived by aggregating the representations of its neighbors, obtained by weighting based on the learned significant weights as follows:

$$Z_i^V = \sum_{k\in N_i}(\gamma_{i,k} \times \vec{h_k}) \tag{6}$$

The structured autoencoder concludes with a structure decoder that uses the embedded representations Z^V as inputs to reconstruct the original node network structure. The reconstruction is achieved through:

$$\hat{A} = \sigma\left(Z^V(Z^V)^T\right) = sigmoid\left(Z^V(Z^V)^T\right) \tag{7}$$

where $sigmoid(\cdot)$ denotes the sigmoid activation function. This step utilizes the inner product between pairs of node embeddings to estimate the probability of a link existing between two nodes:

$$p\left(\hat{A}_{i,j} = 1 | Z_i^V, Z_j^V\right) = sigmoid\left(Z_i^V(Z_j^V)^T\right) \tag{8}$$

The structure autoencoder can learn the structural patterns of the network, facilitating the reconstruction of the network's topology based on the learned embeddings. By integrating attention mechanisms, the autoencoder not only captures the direct relationships between nodes but also effectively learns from the overall structure of the microservice system, enhancing its ability to model complex networks.

2) Attribute Autoencoder: The attribute encoder focuses on analyzing the attribute information in service requests and responses, such as the duration of the request. Using the LSTM-AE model, the attribute encoder can effectively capture dependency relationships in time series data to learn dynamic patterns of service behavior. Compared to traditional Recurrent Neural Networks (RNNs), LSTM can perform better in processing longer sequences thanks to their ability to maintain two states: the cell state (c_t) and the hidden state (h_t), for the current input (X_t) and the previous hidden state (h_{t-1}). The forward propagation in an LSTM cell with a forget gate involves the following equations:

$$Forget\ \ gate: f_t = \sigma_g(W_f X_t + U_f h_{t-1} + b_f)$$
$$Input\ \ gate: i_t = \sigma_g(W_i X_t + U_i h_{t-1} + b_i)$$

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

$$\begin{aligned} Output\ \ gate &: o_t = \sigma_g(W_o X_t \ + \ U_o h_{t-1} \ + \ b_o) \\ Cell\ \ input &: \widetilde{c}_t = \sigma_c(W_c X_t \ + \ U_c h_{t-1} \ + \ b_c) \\ Cell\ \ state\ \ update &: c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t \\ Hidden\ \ state\ \ update &: h_t = o_t \odot \sigma_h(c_t) \end{aligned} \quad (9)$$

Here, we designate d and h as the number of input features and the number of hidden units, respectively the operator $\odot$ the Hadamard product. The vectors i_t, f_t, o_t are the activations for the input, forget, and output gates, respectively. The weight matrices $W_{f,i,o,c} \in R^{h\times d}$ and recurrent weights $U_{f,i,o,c} \in R^{h\times h}$ are learned during training. The bias vectors $b_{f,i,o,c} \in R^h$ are used for each gate and cell state update. The LSTM unit receives an input vector $x_t \in R^d$. The initial values for the cell and hidden states are set to $c_0 = 0$ and $h_0 = 0$, respectively. During the computation process, σ_g represents the sigmoid activation function, σ_c and σ_h refer to the hyperbolic tangent function.

In the attribute encoder, we employ the LSTM-AE model to encode the monitored microservice feature attribute time series data *X*, (including request response time, CPU usage and memory usage) into an attribute embedding hidden layer Z^X, which is very important for TraceDAE. In the decoding phase, the attribute decoder fuses the node embedding Z^A and the attribute embedding Z^X learned by the structure autoencoder, which is used as the input for decoding the original node attributes:

$$\hat{X} = Z^V \left(Z^A\right)^T \quad (10)$$

3) Joint Optimizing: The outputs of the structure autoencoder and the attribute autoencoder are jointly optimized by integrating them into a unified model. This integration is achieved through the calculation of the reconstruction loss score, which evaluates the model's capacity to rebuild the structural and attribute information of a given node accurately. To emphasize the importance of different features, we assign weights to the reconstruction losses associated with them. The primary aim of TraceDAE's training is to reduce the reconstruction losses related to both the network topology and the node attributes:

$$L = \alpha \left\| \left(A - \hat{A}\right) \odot \theta \right\|_F^2 + (1-\alpha) \left\| \left(X - \hat{X}\right) \odot \eta \right\|_F^2 \quad (11)$$

where α denotes the parameter that adjusts the balance between structure and attribute reconstruction. The symbol $\odot$ denotes the Hadamard product. The variables η and θ are defined to penalize the reconstruction error of non-zero elements more heavily, compensating for the presence of missing edges or attributes in empirical datasets:

$$\begin{aligned} \theta_{i,j} &= \begin{cases} 1, \text{ if } A_{i,j} = 0 \\ \theta, \text{ otherwise} \end{cases} \\ \eta_{i,j} &= \begin{cases} 1, \text{ if } X_{i,j} = 0 \\ \eta, \text{ otherwise} \end{cases} \end{aligned} \quad (12)$$

In these definitions, both η and θ are greater than 1, indicating the additional penalties applied to the reconstruction of non-zero elements due to missing edges or attributes.

In TraceDAE, the computational complexity of the structural autoencoder and attribute autoencoder are $O(ND+ED+N^2)$ and $O\ (MD + NM)$, respectively, where N is the number of nodes, E is the number of edges, M is the dimension of attributes, and D is the dimension of embeddings.

D. Anomaly Detection

In the anomaly detection component, we use the learned joint representation from the multi-modal learning stage to detect anomalies in real time. We monitor the system's behavior using real-time data inputs and compare it with the learned normal behavior represented by the joint representation. Any deviations from the learned normal behavior indicate an exception to the prescribed specifications of this template.

Given the significant changes in communication modes during different application stages (including rise, run, and fall stages), our approach responds by dynamically adjusting strategies. Specifically, we recalculate the threshold and optimize the anomaly detection parameters based on the specific stage we are currently in. Taking the upgrade phase as an example, there are significant differences in communication frequency and types between microservices compared to the runtime phase. Therefore, we adjust the time window settings and thresholds immediately when detecting phase transitions to ensure that the model can flexibly respond to these dynamic changes.

In addition, our approach deeply considers the differences between microservices. For microservices that assume different roles (e.g., front-end services, business logic handlers, and back-end services), we recognize that they each exhibit unique communication patterns. To improve the accuracy of anomaly detection, we adopt customized aggregation and anomaly detection methods for each type of microservice. In particular, for microservices with extremely short lifecycles (e.g., processing a single request in milliseconds) [41], traditional detection methods are challenged by a limited observation window. To address this problem, we introduce aggregated metrics and historical pattern analysis based on microservice types to effectively capture and identify anomalous behaviors even in short-lived instances. At the same time, we flexibly adjust the monitoring frequency according to the expected lifetime of the microservices to ensure more efficient anomaly detection within the limited lifecycle.

Finally, we calculate the anomaly score for each trace using the final loss value formula:

$$Anomaly\ \ Score(x_i) = \frac{x_i - \mu_x}{\sigma_x} \quad (13)$$

where x_i is the current data point, μ_x is the mean of the data points, calculated as $\mu_x = \frac{\sum_{i\in N} x_i}{N}$, σ_x is the standard deviation, calculated as $\sigma_x = \frac{\sum_{i\in N} (x_i - \mu_x)^2}{N}$.

The threshold γ is defined as $\mu_x + 3\sigma_x$, where μ_x and σ_x represent the mean and standard deviation, respectively, of historical data. A trace is labeled as an anomaly if its anomaly score exceeds γ. We establish a time window to compute the mean and standard deviation based on historical data during that time period. Setting the time window too small can result in too many false anomalies, while setting it too large will prevent the detection of anomalies. Based on

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

business experience, we set the time window to 30 minutes, which can be manually adjusted during runtime.

E. Root Cause Localization

When an anomaly is detected, TraceDAE analyzes the abnormal trace for the root cause. Each node within the trace represented as N_i, is scrutinized for deviations from normal behavior, focusing on both its structural and attribute characteristics. The anomaly score S_i for node N_i is defined based on the observation that abnormal nodes differ from the majority in terms of structure or attributes. We quantify the reconstruction error from both the network structure and node attribute perspectives:

$$S_i = \alpha \left\| \left(A_i - \hat{A}_i \right) \odot \theta \right\|_F^2 + (1-\alpha) \left\| \left(X_i - \hat{X}_i \right) \odot \eta \right\|_F^2 \quad (14)$$

where A_i and X_i represent the actual network structure and node attributes, respectively, while A_i and X_i denote their predicted values based on the model. The symbols θ and η are weights applied to the network structure and node attributes, respectively. The parameter α balances the importance between structural and attribute discrepancies, and the Frobenius norm ($\| \cdot \|_F$) measures the magnitude of these discrepancies.

Based on the calculated anomaly scores, TraceDAE identifies and ranks the nodes with the highest deviations, labeling the top-k as anomalous. It further correlates these anomalous nodes with their corresponding microservices using the trace feature matrix, thus highlighting the top k microservices likely responsible for the anomalies. This methodology allows for systematic identification and analysis of anomalies within complex microservice architectures, aiding in swift and accurate troubleshooting.

V. Evaluation

A. Dataset

In this study, we evaluate the anomaly detection effectiveness of TraceDAE with datasets generated by two microservice systems. The first dataset, $\mathcal{D}1$, comes from Train-Ticket,[1] a widely used open source microservice benchmarking system. The second dataset, $\mathcal{D}2$, is derived from the production microservice system of a large Internet company, which includes several data sources such as system metrics, user behavior logs, and transaction data. The details of the datasets are provided in Table II, which includes the number of microservice instances, the number of failures that occurred in the microservice instances, and the total number of trace records in each dataset.

1) Train-Ticket Dataset $\mathcal{D}1$: We deploy the microservice system to generate the Train-Ticket dataset. The system is deployed on a Kubernetes cluster of four servers, which consists of three worker nodes running the application services and a master node responsible for coordinating cluster activities and resource allocation, each equipped with a 12-core 2.4 GHz CPU and 12 GB RAM, capable of deploying multiple service instances. We utilize cAdvisor[2] and OpenTelemetry[3] to collect performance metrics and trace data, respectively. Prometheus[4] is used for the persistent storage of collected data, ensuring that all metrics and traces can be efficiently queried and analyzed over time. In addition, to simulate real user requests and workloads, we use Locust[5] to set specific user behavior scripts and request frequencies to generate traffic patterns that are close to real-world scenarios. To obtain a comprehensive sample of anomalies, we inject various types of faults into the train ticket system. As shown in Table III, these faults include scenarios such as high CPU utilization, memory leaks, network congestion, and pod crash loops.

Over the two hours, we collected 242,259 traces, including both normal operations and fault-injected scenarios, leading to the construction of 200 distinct fault conditions across four categories. Of these, 26,751 traces (approximately 11.04%) were impacted by faults. Notably, 11 of these conditions are designed to emulate complex, multi-root cause faults, reflecting the intricate nature of real-world system failures. The fault conditions are distributed as follows: 1.99% is related to high CPU usage, 2.54% is related to memory leaks, and 4.77% is related to network abnormalities, all of which can lead to abnormal service responses. Additionally, 3.48% are related to pod abnormalities, which can cause abnormal service invocation. The average depth of trace data is 6.3 spans, providing rich contextual information for each operation. For ease of reference, we refer to the Train-Ticket dataset as $\mathcal{D}1$.

2) Online Microservice System Dataset $\mathcal{D}2$: Dataset $\mathcal{D}2$ originates from an operational e-commerce microservice system. To ensure the accuracy and reliability of the data, we specifically invite professional operators to annotate the traces in the dataset and to conduct a thorough classification of anomaly types. As a result of their efforts, we collect a total of 1,136,825 trace records. The average depth of trace records is 6.8 spans. Among these, 17,041 traces (representing 1.50% of the total dataset) are identified as faulty. These faulty traces are further subdivided into two types: Service Response Abnormal and Service Response Abnormal. However, due to the sensitivity of the information contained in the dataset, including commercial confidentiality and user privacy, direct public access to the dataset is limited. We are exploring ways to provide access to sanitized versions of the datasets to protect user privacy and comply with confidentiality agreements.

B. Experiment Setup

We have implemented the TraceDAE using a combination of transformer-based attention mechanisms and LSTM-based memory networks. By integrating the advantages of both and fully exploiting their interactions, we are able to provide a more comprehensive and in-depth representation of system behavior. The TraceDAE model has been constructed and

[1] https://github.com/FudanSELab/train-ticket

[2] https://github.com/google/cadvisor

[3] https://opentelemetry.io

[4] https://prometheus.io

[5] https://locust.io

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE DETAILS OF DATASETS

Dataset	Microservice Cases	Trace Records	Service Response Abnormal Trace Records	Service Invocation Abnormal Trace Records	Abnormal Trace Records	The Average Depth of Trace Record	Abnormal Rate
$\mathcal{D}1$	39	242259	22531	8420	26751	4.2	11.04%
$\mathcal{D}2$	18	1,538,382	24,070	17,041	29,431	6.8	1.91%

TABLE III
THE TYPES OF FAULT INJECTION AND DETAILED

Fault types	Example Descriptions	Abnormal Type	Abnormal Records	Abnormal Rate
CPU abnormal	Includes issues like high CPU usage, CPU throttling,overheating, and processor failures. These faults can slow down or halt processes and affect system stability.	Service Response Abnormal	4815	1.99%
Memory abnormal	Involves problems such as memory leaks, insufficient RAM, memory corruption, and access violations, leading to crashes, slow performance, or system instability.	Service Response Abnormal	6153	2.54%
Network abnormal	Covers network congestion, lost connectivity, high latency, packet loss, and configuration errors, affecting communication and data transfer between systems.	Service Response Abnormal	11563	4.77%
Pod abnormal	Includes issues like crash loops, failed deployments, resource limits, and connectivity problems, impacting containerized applications.	Service Invocation Abnormal	8420	3.48%

trained within the PyTorch framework. The source[6] is publicly available.

To optimize the TraceDAE model, we employ the Adam optimizer, chosen for its adaptive learning rate capabilities, making it well-suited for our complex data. The learning rate is set to 0.001, balancing speed and accuracy, and a batch size of 32 is found optimal for our GPU memory constraints. We set the number of epochs to 100 and use early stopping if the validation loss does not improve for 10 consecutive epochs. The embedding dimension of the two datasets is set to 128, and the parameters (α, η, θ) are set to (0.1, 5, 40) and (0.1, 3, 10). The experiments are conducted on a high-performance computer equipped with NVIDIA GeForce RTX 4050 Laptop GPU, which accelerates the speed of model training.

C. Performance Metrics

To evaluate the performance of our approach, we use three common evaluation metrics for anomaly detection: precision, recall, and F1-score. Trace anomaly detection is approached as a binary classification task for each trace, where the F1-score serves as a critical metric for evaluating performance. The F1-score is derived from the counts of true positive samples (TP), false positive samples (FP), and false negative samples (FN) produced by the model tasked with identifying failures. In addition, we expand our evaluation by incorporating additional metrics such as ROC-AUC and precise recall curves to offer a more comprehensive assessment of TraceDAE's effectiveness. To evaluate the efficiency of our method, we use training time and inference time as efficiency evaluation metrics.

- *Precision:* the proportion of true positive predictions among all positive predictions made by the model., which can be calculated as: $Precision = \frac{TP}{TP+FP}$.
- *Recall:* the proportion of true positive predictions among all actual positive instances, which can be calculated as: $Recall = \frac{TP}{TP+FN}$.
- *F1-score:* the harmonic mean of precision and recall, providing a balance between the two metrics, which can be calculated as: $F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$.

To evaluate the precision of TraceDAE in root cause localization, we use the common evaluation metric Top-k accuracy (A@k).

- *Top-k Accuracy (A@k):* This metric measures the likelihood that the actual root causes are among the top-k identified candidates. For our evaluation, we select k values of 1, 2, and 3 to cover a range of scenarios.

Based on the above metrics, we can evaluate the effectiveness of TraceDAE and comparative baseline SOTA(State of The Arts) methods in anomaly detection and root cause localization in microservice systems.

D. Baseline Methods

To comprehensively evaluate the performance of TraceDAE in anomaly detection, we compare it against six SOTA trace anomaly detection methods, including the latest deep learning and machine learning-based methods, as well as solutions designed specifically for microservice systems.

- FSA [25]. Uses a finite-state automaton to capture invocation dependencies and time consumptions, flagging traces as anomalous if rejected by the FSA. This approach is a classical method widely recognized for its effectiveness in trace anomaly detection.
- MultimodalTrace [26]. Organizes microservices and their time consumptions in sequences modeled using LSTM-VAE. This method optimizes node anomaly scores uniquely, aiming for maximization rather than aggregation.
- TraceAnomaly [27]. Traces are encoded into vectors, where each dimension represents the time consumption of a specific "path". Negative Log Likelihoods (NLLs) are employed as anomaly scores, providing a quantitative measure of deviation from expected behavior.

[6]https://github.com/hellolijj/traceDAE.git

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
THE EFFECTS OF TRACEDAE IN COMPARISON WITH BASELINE METHODS ON TWO DATASETS

	$\mathcal{D}1$			$\mathcal{D}2$		
	precision	recall	F1-score	precision	recall	F1-score
FSA	0.835	0.823	0.829	0.851	0.849	0.85
MultimodalTrace	0.72	0.854	0.781	0.92	0.819	0.867
TraceAnomaly	0.766	0.711	0.737	0.823	0.85	0.836
DeepTraLog	0.838	0.725	0.777	0.781	0.675	0.724
TraceCRL	0.823	0.704	0.759	0.861	0.755	0.805
TraceVAE	0.928	0.832	0.877	0.915	0.814	0.821
TraceDAE	**0.971**	**0.983**	**0.977**	**0.95**	**0.971**	**0.96**

- DeepTraLog [32]. The complex calling structure between traces and time of service logs is processed by the Deep SVDD model for unified anomaly detection. At the same time, it provides a user feedback mechanism to enable dynamic adjustment of weights over time.
- TraceCRL [34]. It employs a representation learning approach that combines contrastive learning with graph neural networks to incorporate graph-structured information into downstream trace analysis tasks.
- TraceVAE [33]. The approach utilizes a novel model to identify anomalies in traces. It captures the trace characteristics using a dual-variable graph VAE model and detects anomalies through NLL. Additionally, it incorporates techniques aimed at minimizing the entropy gap.

In implementing the Multimodal-Trace method, we employ two LSTM networks to process the input data and equally distributed the weights of the two loss functions, setting each weight to 0.5, to ensure the network could adequately learn the multifaceted characteristics of traces during training. For the TraceAnomaly method, we set the dimensionality of the latent variable z to 10, which aids in capturing the deep structures and anomaly patterns within trace data. Considering that the original design of DeepTraLog requires processing both log and trace data, and our current dataset contains only trace information, we substitute the event vector inputs from the original method with the feature matrix of TPG to adapt to our data characteristics and fully leverage DeepTraLog's potential in trace anomaly detection. For the TraceVAE method, we set the parameter $K = 5$ to represent the structural variable Z, and $K = 10$ to represent the time variable $Z2$. These adjustments are aimed at ensuring all methods can operate in their optimal state on the same dataset, thus allowing for a fair comparison of their anomaly detection performance.

To evaluate the precision of TraceDAE in root cause localization, we compare it with three recently proposed methods: MEPFL [17], TraceAnomaly [19], MicroRank [36], and TraceRCA [38]. We describe the parameter settings for all methods to ensure a fair comparison. For MicroRank, we set the damping factor d to 0.85 and maintain the preference vector weight φ to default value 0.5. Similarly, for TraceRCA, we utilize the recommended default values for parameters δ_{ad},δ_{fs} and δ_{spt} which are 1, 0.1 and 0.1, respectively.

E. Result

To evaluate the performance of anomaly detection methods, we compute and compare the average precision, recall, and optimal F1-score as presented in Table IV. Using these metrics, we compare the performance of TraceDAE against five baseline approaches across two datasets. The results showcase the superior performance of TraceDAE in detecting anomalies within microservice systems, evidenced by its high precision (0.971), recall (0.935), and F1-score (0.953). Compared to the best-performing baseline method, TraceDAE achieves higher optimal F1-score by 17.9% and 10.7% on the respective datasets, further confirming its effectiveness in the field of microservice anomaly detection.

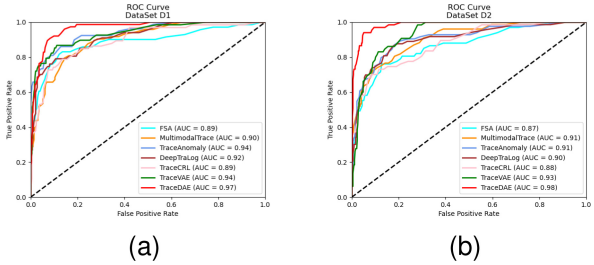


Fig. 4. The ROC Curve of TraceDAE in comparison with baseline methods on two datasets.

Among the evaluated methods, TraceAnomaly and MultimodalTrace exhibit relatively weaker performance in terms of F1-score, primarily due to their reliance only on trace data of microservice systems (Trace data mainly focuses on the service invocation relationships and service response time indicators), while ignoring key performance monitoring metrics in microservice systems, such as CPU usage and memory usage, which are essential for identifying abnormal resource usage. In contrast, FSA demonstrates stable and moderate performance across all datasets but requires extensive parameter adjustments for each target microservice system. TraceCRL, despite its slightly inferior ability to detect call structure anomalies, surpassed TraceAnomaly in overall performance. DeepTraLog benefits from its deep SVDD learning rule feature representation and amine hyperspheres in preprocessing all datasets, achieving a good performance. TraceVAE has achieved good performance because it can effectively detect structural anomalies in Trace, but it cannot effectively detect a small number of attribute anomalies.

In addition, we also use precision-recall curves to compare the anomaly detection capabilities of all methods. The area under the curve (AUC) demonstrates the ability of different methods to identify anomalies. The larger the AUC value, the higher the accuracy of this method. As shown in Figure 4, TraceDAE performs well on all datasets.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

TABLE V
THE ACCURACY OF ROOT CAUSE LOCALIZATION IN COMPARISON WITH DIFFERENT METHODS

	$\mathcal{D}1$			$\mathcal{D}2$		
	A@1	A@2	A@3	A@1	A@2	A@3
MEPFL	0.43	0.48	0.53	0.45	0.5	0.56
MicroRank	0.61	0.7	0.78	0.66	0.72	0.81
TraceAnomaly	0.66	-	-	0.69	-	-
TraceRCA	0.71	0.79	0.84	0.75	0.82	0.88
TraceDAE	**0.8**	**0.89**	**0.95**	**0.83**	**0.91**	**0.96**

Besides anomaly detection, we also compare the accuracy of TraceDAE in root cause localization with the baseline methods of two datasets, as shown in Table V. We observe that the accuracy of TraceDAE in root cause localization is better than the baseline method. Notably, TraceDAE exhibits exceptional performance in precision at the top rank (A@1), which reflects its robust capability to accurately locate root cause of the primary anomalous microservice.

Additionally, we evaluated the training and inference efficiency of each method. TraceDAE demonstrated superior performance, with detailed timings presented in Table VI. MulitmodaleTrace and TraceCRL exhibited relatively brief training times of 20ms and 24.6ms for the dataset A, and 7.8ms and 23.4ms for the dataset B, due to their efficient handling of sequence data. TraceAnomaly, which transforms traces into Structured Trace Vectors (STVs) for the detection of response time and call path anomalies, has training times of 49.2ms and 42.4ms for the respective datasets. Its inference times s are at 19.2ms and 17.4ms, indicating a more resource-intensive approach. DeepTraLog, which processes a significant amount of simulated log information, exhibited the longest training times of 158.5ms and 135.1ms, respectively. Other methods such as FSA and TraceVAE also showed varying degrees of efficiency. FSA's training time is not available, but it had one of the shortest inference times s of 0.9ms and 0.7ms. TraceVAE required 43.1ms and 30.4ms for training, with inference times s of 8.2ms and 7.6ms. TraceDAE introduced the Service Trace Graph (STG) method to present service trace. This method significantly reduced the time required, making it the shortest training time of all methods, requiring only 6.1ms and 4.3ms for two separate datasets. Its inference times s are also minimal, at just 0.6ms and 0.5ms, respectively.

F. Ablation Study

In this section, we evaluated the impact of various components within our proposed TraceDAE model by systematically replacing or disabling specific modules.

Initially, in order to explore the impact brought by the STG module, we replace the STG in TraceDAE with the STV(Service Trace Vector) in TraceDAE. The results of the experiment are shown in Table VII, where the method using STV shows a decrease in both precision and recall, which indicates that STG outperforms STV in extracting relevant features from the trace data. Meanwhile, the experiment also shows that the execution time of using STV is significantly longer than TraceDAE, which further proves that the using of STG has higher efficiency in processing trace data compared to using STV. This experiment emphasizes the key role of STG in TraceDAE.

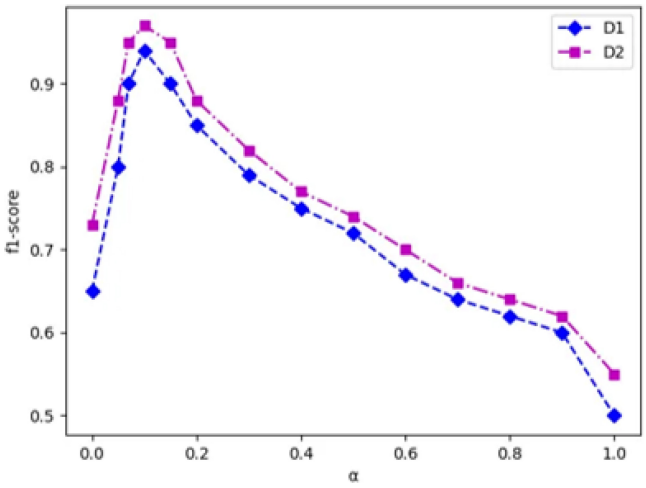


Fig. 5. Parameter sensitivity experiment results about α.

In addition, we evaluated the impact of the noise reducing step. As shown in Table VII, removing the noise reduction step (w/o DBSCAN) resulted in lower precision and recall, suggesting that the noise reduction step is critical for minimizing false alarms by preventing the model from learning from anomalous trajectory patterns.

Finally, we explore the impact of each autoencoder by removing the Structure-AE and Attribute-AE modules from TraceDAE, respectively. Table VIII presents the best F1-scores for service response abnormal (SRA), service invocation abnormal (SIA), and overall abnormal for two approaches across two datasets. Experimental results indicate a significant decrease in SRA performance upon the removal of Attribute-AE, and a notable reduction in SIA performance upon the removal of Structure-AE. This suggests that the LSTM component of Attribute-AE excels at detecting response time and other performance anomalies, while the GAT component of Structure-AE demonstrates a higher sensitivity to service invocation path anomalies. The combined usage of Attribute-AE and Attribute-AE enhances the overall anomaly detection capability.

G. Parametric Analysis

In this section, we investigate the impact of the parameter α on the average optimal F1-score of TraceDAE. The results from the parameter sensitivity experiments presented in Fig. 5 reveal that when α is set to 0 or 1, relying only on Attribute-AE or Structure-AE results in a decrease in performance, which highlights the synergistic role of Attribute-AE and Structure-AE in trace-based anomaly detection in microservice systems. As α increases from 0.0 to 1.0, we observe that the average F1-score of TraceDAE peaks when α set to 0.1. Further increasing α affects the selection of the anomaly score threshold, resulting in a performance degradation. Consequently, we set α to 0.1.

In the experimental process of anomaly detection, the threshold parameter γ has a significant impact on the

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
THE TRAINING AND INFERENCE TIME OF TRACEDAE IN COMPARISON WITH BASELINE METHODS ON TWO DATASETS

	$\mathcal{D}1$		$\mathcal{D}2$	
	Training Time(ms)	Inference Time(ms)	Training Time(ms)	Inference Time(ms)
FSA	N/A	0.9	N/A	0.7
MultimodalTrace	20	1.6	17.8	1.3
TraceAnomaly	49.2	19.2	42.4	17.4
DeepTraLog	158.5	1.8	135.1	1.7
TraceCRL	24.6	6.3	23.4	5.5
TraceVAE	43.1	8.2	30.4	7.6
TraceDAE	**6.1**	**0.6**	**4.3**	**0.5**

TABLE VII
THE EFFECTS OF TRACEDAE IN COMPARISON WITH DIFFERENT ALTERNATIVE MODEL ON TWO DATASETS

	$\mathcal{D}1$				$\mathcal{D}2$			
	precision	recall	F1-score	Inference time(ms)	precision	recall	F1-score	Inference time(ms)
using STV	0.915	0.903	0.909	0.64	0.901	0.829	0.864	0.58
w/o DBSCAN	0.921	0.915	0.918	N/A	0.917	0.804	0.857	N/A
TraceDAE	**0.964**	**0.968**	**0.966**	**0.6**	**0.921**	**0.933**	**0.927**	**0.5**

TABLE VIII
THE EFFECTS OF TRACEDAE IN COMPARISON WITH DIFFERENT ALTERNATIVE AE ON TWO DATASETS

	$\mathcal{D}1$(F1-core)			$\mathcal{D}2$(F1-core)		
	SRA	SIA	Overall	SRA	SIA	Over
w/o Structure-AE	0.561	0.818	0.876	0.516	0.835	0.861
w/o Attribute-AE	0.83	0.502	0.884	0.813	0.421	0.846
TraceDAE	**0.861**	**0.872**	**0.965**	**0.875**	**0.912**	**0.948**

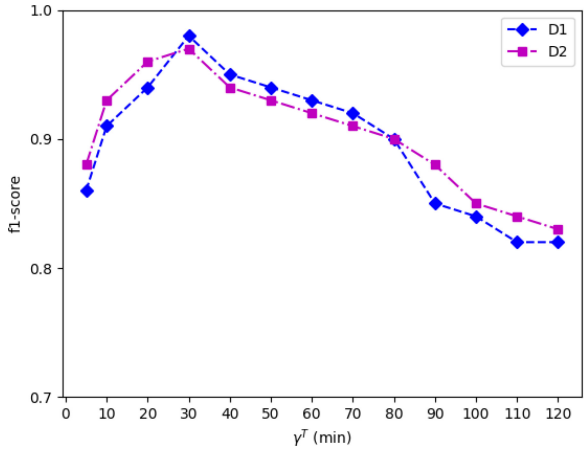


Fig. 6. Parameter sensitivity experiment results about γ^T.

performance of the detection results. The threshold γ is defined as $\mu_x + 3\sigma_x$ where μ_x and σ_x represent the average and standard deviation of a time window γ^T, respectively. To determine the optimal value of γ, we designed a series of parameter experiments for the time window γ^T. As shown in Fig 6, the experimental results indicate that as γ^T increases, the detection accuracy gradually improves. At $\gamma^T = 30$ minutes, the F1-score reached its highest value. Therefore, this article selects $\gamma^T = 30$ minutes as the optimal threshold for anomaly detection.

VI. CONCLUSION

In this paper, we propose a novel approach, TraceDAE, for trace-based anomaly detection in microservice systems. By constructing a service trace graph (STG), we effectively illustrate service invocation relationships and performance metrics. Utilizing the STG, we have designed a dual autoencoder framework to jointly model the interactions between different data sources using the attention mechanism to capture complementary information between different data sources. Experimental results show that the STG representation makes our approach more efficient in model training and anomaly detection, and the dual autoencoder framework makes TraceDAE effective for anomaly detection and root cause localization.

In the future, we will explore the relationship between multimodal data from microservice systems to efficiently detect anomalies in microservice systems and locate the root cause of failures.

ACKNOWLEDGMENT

The authors would like to thank engineers such as Kai Shi from the Cloud Native team of AliCloud Inc. for the enlightening discussions and exchanges provided to this paper. They also thank the anonymous reviewers for providing valuable feedback and suggestions that greatly improved the quality of this paper.

REFERENCES

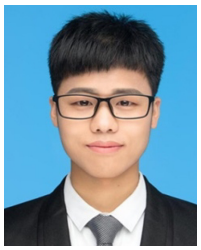
[1] Z. Liu, N. Japkowicz, R. Wang, Y. Cai, D. Tang, and X. Cai, "A statistical pattern based feature extraction method on system call traces for anomaly detection," *Inf. Softw. Technol.*, vol. 126, Oct. 2020, Art. no. 106348.

[2] N. Singh et al., "Load balancing and service discovery using docker swarm for microservice based big data applications," *J. Cloud Comput.-Adv. Syst. Appl.*, vol. 12, no. 1, p. 4, Jul. 2023.

[3] J. Diaz-De-Arcaya, A. I. Torre-Bastida, G. Zarate, R. Minon, and A. Almeida, "A joint study of the challenges, opportunities, and Roadmap of MLOps and AIOps: A systematic survey," *ACM Comput. Surv.*, vol. 56, no. 4, pp. 1–30, Apr. 2024.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.

[4] P. Jamshidi, C. Pahl, N. C. Mendonca, J. Lewis, and S. Tilkov, "Microservices: The journey so far and challenges ahead," *IEEE Softw.*, vol. 35, no. 3, pp. 24–35, May/Jun. 2018.
[5] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45TH Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1750–1762.
[6] F. Zanettin. "Didi system glitch lasts 12 hours, with company apologizing for service breakdown." 2023. [Online]. Available: https://english.visitbeijing.com.cn/article/4FZquTLvAMl
[7] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 1–39, Apr. 2023.
[8] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proc. 31ST ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2023, pp. 553–565.
[9] S. Zhang et al., "Robust failure diagnosis of microservice system through multimodal data," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 3851–3864, Nov./Dec. 2023.
[10] K. Zhang, C. Zhang, X. Peng, and C. Sha, "PUTraceAD: Trace anomaly detection with partial labels based on GNN and PU learning," in *Proc. IEEE 33RD Int. Symp. Softw. Reliab. Eng. (ISSRE)*, 2022, pp. 239–250.
[11] Z. Xie et al., "From point-wise to group-wise: A fast and accurate microservice trace anomaly detection approach," in *Proc. 31ST ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2023, pp. 1739–1749.
[12] W. Meng et al., "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 4739–4745.
[13] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and unified deep learning based log anomaly detection for diverse faults," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2020, pp. 92–103.
[14] C. Zhang, X. Wang, H. Zhang, H. Zhang, and P. Han, "Log sequence anomaly detection based on local information extraction and globally sparse transformer model," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4119–4133, Dec. 2021.
[15] J. Chen et al., "TraceGra: A trace-based anomaly detection for microservice using graph deep learning," *Comput. Commun.*, vol. 204, pp. 109–117, Apr. 2023.
[16] P. Las-Casas, J. Mace, D. Guedes, and R. Fonseca, "Weighted sampling of execution traces: Capturing more needles and less hay," in *Proc. ACM SoCC*, 2018, pp. 326–332.
[17] X. Zhou et al., "Latent error prediction and fault localization for microservice applications by learning from system trace logs," in *Proc. 27th ACM ESEC/FSE*, 2019, pp. 683–694.
[18] H. Fan, F. Zhang, and Z. Li, "Anomalydae: Dual autoencoder for anomaly detection on attributed networks," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2020, pp. 5685–5689.
[19] X. Zhou et al., "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.
[20] S. Zhang et al., "Efficient and robust trace anomaly detection for large-scale microservice systems," in *Proc. IEEE 34th Int. Symp. Softw. Reliab. Eng. (ISSRE)*, 2023, pp. 69–79.
[21] J. Huang, Y. Yang, H. Yu, J. Li, and X. Zheng, "Twin graph-based anomaly detection via attentive multi-modal learning for microservice system," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2023, pp. 66–78.
[22] Z. He et al., "A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023.
[23] C. Zhang, T. Zhou, Q. Wen, and L. Sun, "TFAD: A decomposition time series anomaly detection architecture with time-frequency analysis," in *Proc. 31ST ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2022, pp. 2497–2507.
[24] L. Yang, J. Li, K. Shi, S. Yang, Q. Yang, and J. Sun, "MicroMILTS: Fault location for microservices based mutual information and LSTM autoencoder," in *Proc. 23rd Asia-Pac. Netw. Oper. Manag. Symp. (APNOMS)*, 2022, pp. 73–78.
[25] Q. Fu, J. G. Lou, Y. Wang, and J. Li, "Execution anomaly detection in distributed systems through unstructured log analysis," in *Proc. 9th IEEE Int. Conf. Data Min.*, 2009, pp. 149–158.
[26] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD)*, 2019, pp. 179–186.
[27] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2020, pp. 48–58.
[28] Z. Zhang, M. K. Ramanathan, P. Raj, A. Parwal, T. Sherwood, and M. Chabbi, "{*CRISP*}: Critical path analysis of {large-scale} microservice architectures," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2022, pp. 655–672.
[29] L. Meng, F. Ji, Y. Sun, and T. Wang, "Detecting anomalies in microservices with execution trace comparison," *Future Gener. Comput. Syst.*, vol. 116, pp. 291–301, Mar. 2021.
[30] T. Wang, W. Zhang, J. Xu, and Z. Gu, "Workflow-aware automatic fault diagnosis for microservice-based applications with statistics," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2350–2363, Dec. 2020.
[31] Y. Xu et al., "TraceLingo: Trace representation and learning for performance issue diagnosis in cloud services," in *Proc. IEEE/ACM Int. Workshop Cloud Intell. (CloudIntell.)*, 2021, pp. 37–40.
[32] C. Zhang et al., "DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. ACM/IEEE 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, pp. 623–634.
[33] Z. Xie et al., "Unsupervised anomaly detection on microservice traces through graph VAE," in *Proc. ACM Web Conf.*, 2023, pp. 2874–2884.
[34] C. Zhang et al., "TraceCRL: Contrastive representation learning for microservice trace analysis," in *Proc. 30th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2022, pp. 1221–1232.
[35] D. Liu et al., "MicroHECL: High-efficient root cause localization in large-scale microservice systems," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. Softw. Eng. Pract. (ICSE-SEIP)*, 2021, pp. 338–347.
[36] G. Yu et al., "MicroRank: End-to-end latency issue localization with extended spectrum analysis in Microservice environments," in *Proc. Web Conf.*, 2021, pp. 3087–3098.
[37] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Proc. 16th Int. Conf. Service-Oriented Comput. (ICSOC)*, 2018, pp. 3–20.
[38] Z. Li et al., "Practical root cause localization for microservice systems via trace analysis," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Service (IWQOS)*, 2021, pp. 1–10.
[39] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. Nat. Conf. Aritif. Intell.*, 1999, pp. 226–231.
[40] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2014, pp. 701–710. [Online]. Available: https://doi.org/10.1145/2623330.2623732
[41] M. M. Ghorbani, F. F. Moghaddam, M. Zhang, M. Pourzandi, K. K. Nguyen, and M. Cheriet, "Malchain: Virtual application behaviour profiling by aggregated microservice data exchange graph," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, 2020, pp. 41–48.



Junjun Li received the B.S. degree in network engineering from the School of information and electronic engineering, Zhejiang Gongshang University, Hangzhou, China, in 2018, and the master's degree in software engineering from the School of Software Technology, Zhejiang University, Hangzhou, in 2020. He is currently pursuing the Ph.D. degree with the School of Computer Science, Wuhan University. His research interests include anomaly detection and root cause localization in microservice systems.



Shi Ying is currently a Professor with the School of Computer Science, Wuhan University, China. His main research interests include software engineering, intelligent operation and maintenance management, cloud computing and cloud service software, and artificial intelligence.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.



Tiangang Li is currently pursuing the Ph.D. degree with the School of Computer, Wuhan University, China. His research interests include cloud computing, software engineering, and resource management and reinforcement learning.



Xiangbo Tian is currently pursuing the Ph.D. degree with the School of Computer, Wuhan University, China. His research interests include cloud computing, software engineering, causal learning, and representation learning.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:36 UTC from IEEE Xplore. Restrictions apply.